

Student: Vibhuti Uttarde | 3rd Year Diploma | Information Technology | Vidyalkar Polytechnic | June 2026

Day 1 — Debugging Discipline

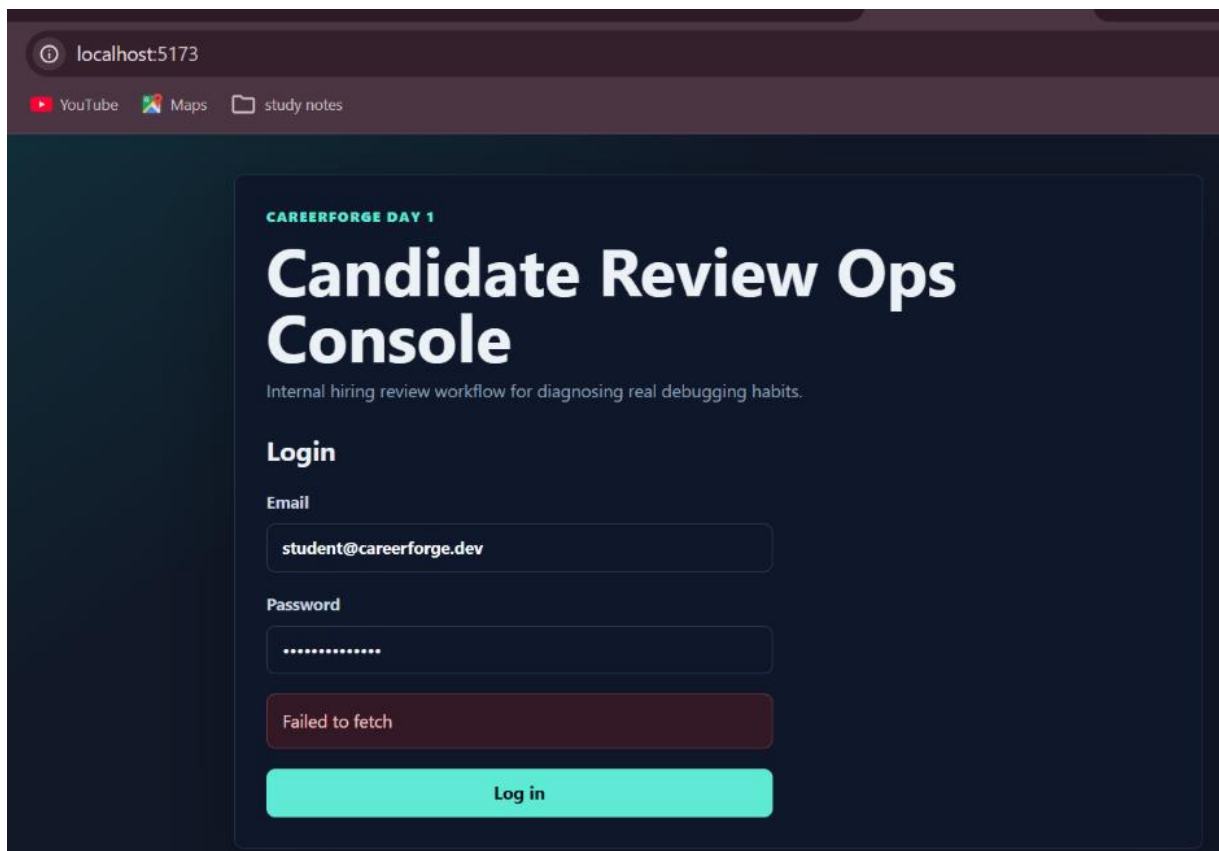
Strongest evidence:

- Traced 4 stacked bugs across backend, auth, and frontend layers using netstat, Network tab, and direct source-code reading — not guesses or AI output.
- Rejected the founder's assumption that it was a small frontend issue; the JWT expiry bug was in backend auth.js and would have been missed entirely if the investigation stayed on the frontend.
- All four root causes fixed in a single commit (3fba0f8): port conflict → port 3001, wrong env URL → correct port, 30s token expiry → 2h, Content-Type header override in api.js → fixed.

Evidence — BEFORE fixes:



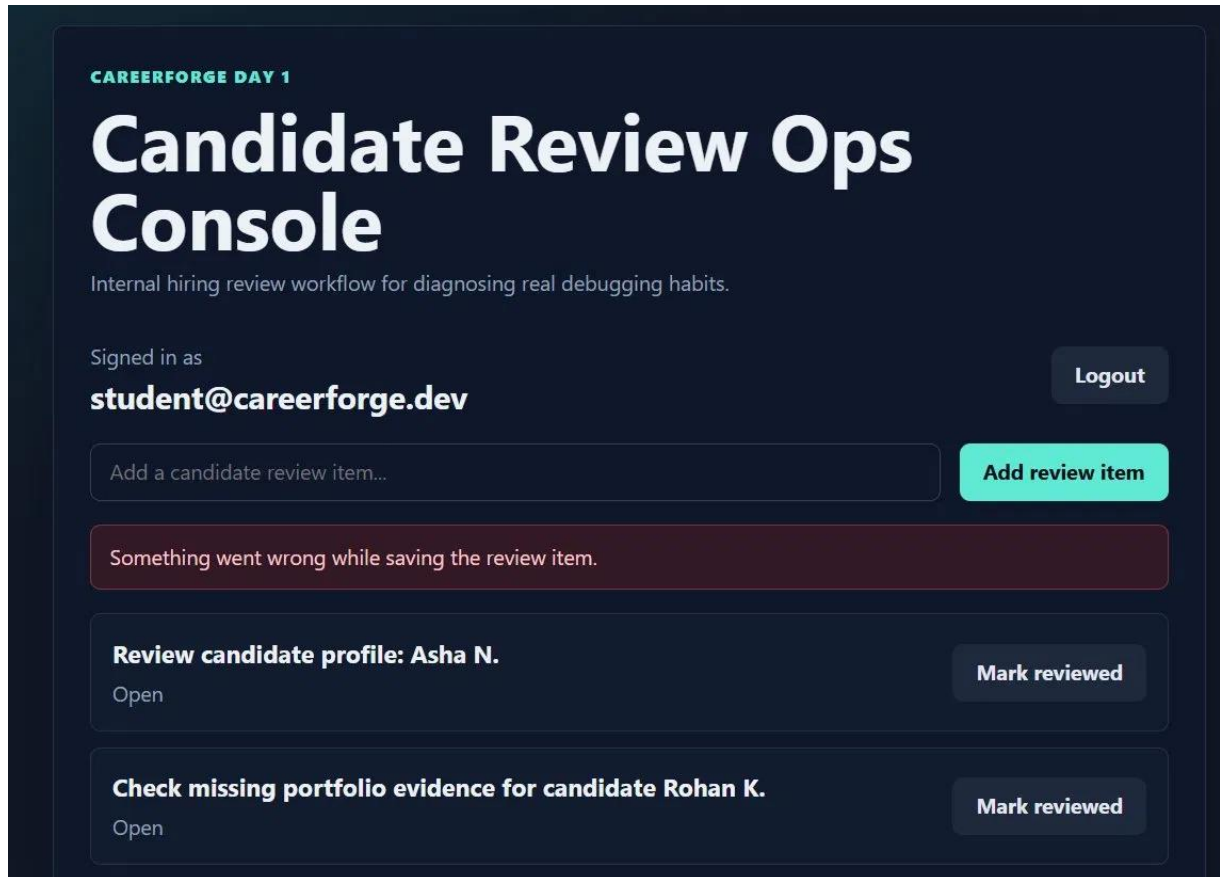
BEFORE: frontend/.env pointing to wrong port 8000 — all requests failed before reaching backend



BEFORE: Login shows 'Failed to fetch' — frontend cannot reach backend at all

```
const token = jwt.sign(  
  { email: TEST_USER.email, name: TEST_USER.name },  
  process.env.JWT_SECRET,  
  { expiresIn: "30s" }  
);
```

BEFORE: auth.js showing expiresIn: '30s' — token dies 30 seconds after login



BEFORE: App loaded but 'Something went wrong while saving the review item' — Content-Type bug causing 400 Bad Request

Evidence — AFTER fixes:

```

16
17     const token = jwt.sign(
18       { email: TEST_USER.email, name: TEST_USER.name },
19       process.env.JWT_SECRET,
20       { expiresIn: "2h" }
21     );
22

```

AFTER: expiresIn changed to '2h' — session stays active throughout review



AFTER: Full workflow working — new review item added, marked Completed, Reopen button visible

```

● PS C:\Users\HP\OneDrive\Desktop\day1\day-01-debugging-investigation> git log --oneline
744a3a5 (HEAD -> day-1-reality-check, origin/day-1-reality-check, main) verification: full workflow te
sted - login, add item, mark reviewed all pass
3fba0f8 fix: JWT expiry 30s to 2h, port 3001, Content-Type header bug in api.js
b40ecab diagnosis: reproduced port conflict, 401 unauthorized, 400 bad request - documented evidence
3176e2c (origin/main, origin/HEAD) docs: clarify student outcomes and case materials
82a2582 chore: publish day 1 challenge package
○ PS C:\Users\HP\OneDrive\Desktop\day1\day-01-debugging-investigation>

```

AFTER: git log showing commit 3fba0f8 with all fixes and verification commit 744a3a5

What this proves: I read source code directly, tested every hypothesis with evidence, and didn't stop at the first fix. The founder was wrong — bugs were spread across backend config, auth logic, and the frontend API layer.

Day 2 — Systems Reasoning

Strongest evidence:

- Identified PostgreSQL write latency (3,500–7,800ms, 14–31x normal) as the single origin failure from log timestamp 10:01:10 — before every downstream effect appeared in the logs.
- Evaluated 4 competing hypotheses and rejected 3 (notification queue, Redis cache, recruiter double-click) as downstream symptoms, each with specific log-line evidence pointing back to the DB write latency.

- Proposed pausing the review sprint to break the retry feedback loop — addressing the root cause without any code change in the first 15 minutes.

What this proves: I can read a log sequence causally, separate symptoms from origin failure, and act on the right lever under time pressure.

Day 3 — Readiness Verification

Strongest evidence:

- Declared the critical workflow before touching anything: Recruiter selects candidate → clicks Analyze Resume → sees score and signals → updates candidate status → dashboard reflects the result.
- Found 3 cascading .env blockers on the exact same code path: ENABLE_RESUME_ANALYSIS=false, empty AI_PROVIDER, wrong RESUME_STORAGE_PATH pointing to a directory that didn't exist.
- Blocked the demo with evidence — health check returning 200 and login working were not proof the workflow worked. Fixed all 4 issues with .env only, no source code changed.

What this proves: I don't use liveness as a proxy for readiness. I verify the declared workflow end-to-end before calling anything ready.

Day 4 — Ambiguity Ownership

Strongest evidence:

- Identified 5 open ambiguities in a single-sentence founder request before writing any code — including a compliance risk buried in 'let them shortlist anyone' (eligibility bypass = different risk to score preference override).
- One clarification question unlocked 4 answers simultaneously: eligibility rule type, reason requirement, permission scope, and audit coverage needed.
- Rejected the AI's full override module (role-based permissions, approval workflow, audit dashboard, bulk override, candidate notifications) as wrong scope for a two-week pilot with a small recruiter cohort.

What this proves: I surface hidden compliance risks before they become production defects. I scope to what is safe and minimal — not what is complete.

Day 5 — AI Trust Judgment

Strongest evidence:

- Blocked 3 of 6 AI recommendations with exact failure sequences: Redis atomic decrement (not durable — restart resets counter, same unit sells twice with Postgres never updated), 60s reconciliation (accepts inconsistency as design, dozens oversell before it runs), service split (distributed transaction failure with no safe rollback for a small team in one week).
- Modified recommendation 2 — backend idempotency keys are correct pattern but must insert key + checkout record atomically with a unique constraint on (user_id, idempotency_key). As written by AI, race condition still creates two sessions.
- Identified the exact failure sequence for the Redis crash scenario: stock=1, A gets 0 (proceeds), B gets -1 (blocked), Redis crashes before async write, Postgres still shows stock=1, Redis reinitialises, C gets 0 (proceeds) — A and C both have paid orders for the same unit.

What this proves: I evaluate AI output as hypotheses requiring evidence and failure-sequence testing, not as instructions to follow.

Day 6 — Pressure Prioritization

Strongest evidence:

- Paused checkout at T+01 before full harm was quantified — because the downside of waiting (duplicate charges compounding) was worse than the downside of acting with incomplete information (10-minute revenue pause).

- Kept engineering on the root cause (webhook idempotency) not the dashboard — analytics being broken doesn't charge anyone twice. Duplicate charges 0→62 and webhook lag 20s→720s confirmed the correct diagnosis.
- Modified AI rollback recommendation at T+07: full rollback risked inventory drift from queued in-flight events (confirmed by Nikhil at T+08). Narrow refund scope (confirmed duplicate IDs only) protected Finance from false-positive chargebacks.

What this proves: I prioritise by harm severity, hold my position under founder pressure with stated reasons, and modify AI recommendations using information the AI doesn't have.

Biggest Mistake This Week

What I got wrong:

Trusting AI suggestions without independently verifying them first. I assumed AI would not give me wrong suggestions. Day 5 proved otherwise — the Redis plan used real commands, real terminology, and sounded authoritative. I had to work backwards through the failure sequence to find the flaw. That reconstruction should have been my first move, not a recovery move.

What I learned:

AI optimises for completeness, not safety. It doesn't know the team size, the timeline, the specific infrastructure failure that already happened, or the pilot scope. I have to bring that context. AI's job is to suggest. My job is to judge.

What I would do differently now:

Before accepting any AI recommendation, ask: what is the failure sequence if this is wrong? If I can't answer that in two sentences, I haven't evaluated it — I've forwarded it.

Strongest Capability

Capability: Evidence-based root cause analysis — tracing failures to their origin using logs, source code, and metrics rather than accepting surface symptoms or authority framing.

Evidence: Demonstrated across Days 1, 2, 3, and 6. In every case the founder or AI pointed at a visible symptom. In every case I traced the evidence chain to the actual origin and acted on that instead.

Weakest Capability

Capability: AI trust judgment — I had a default assumption that AI suggestions would be correct, which meant I was sometimes solving problems the AI created rather than preventing them.

Evidence: Day 5 — the Redis decrement plan required manual failure-sequence reconstruction to identify the flaw. That should have been a first-move habit, not a recovery move.

Next 30-Day Engineering Growth Focus

Focus 1: Not anchoring on founder/authority framing — writing down the assumption being accepted before starting any investigation.

Focus 2: Verifying end-to-end workflows fully before declaring anything ready — not stopping at the component that was changed.

Daily behaviour: Before every debugging session or feature decision, write one sentence declaring the workflow I am actually testing and run it before reading anyone else's framing of the problem.